

Jolt Proofs, Verified On-Chain: A Full zkVM Verifier in Solidity

We built a complete on-chain verifier for [Jolt](#), a sumcheck-based zkVM for RISC-V. The verifier checks every stage of a Jolt proof in Solidity — from Spartan R1CS satisfaction through polynomial opening — and is live on Berachain Bepolia today.

A production-scale SHA3 proof (8,192-step execution trace, 9-round Dory commitment scheme) verifies on-chain in a single transaction for **5.3M gas**.

Why This Is Hard

Most zkVM verifiers deployed on-chain today use a single SNARK (Groth16 or PLONK) that wraps the entire verification circuit. This works but requires the prover to generate a monolithic proof over everything — including the Fiat-Shamir hash computations that dominate constraint count.

Jolt’s verification has 8 stages:

1. **Spartan outer sumcheck** — R1CS constraint satisfaction
2. **Product virtualization** — 5 batched sumcheck instances (RAM, registers, lookups, output)
3. **Instruction lookup read-RAF checking**
4. **Bytecode read-RAF checking**
5. **Register read-write checking** (4 instances)
6. **RAM read-write + lookup RA checking** (3 instances)
7. **Claim reductions** (9 instances covering advice, Hamming weight, increments, registers, RAM)
8. **Dory polynomial opening** — pairing-based commitment verification

Stages 1-7 are algebraic: field arithmetic and Fiat-Shamir hashing. Stage 8 involves elliptic curve pairings. The key insight is that these have very different cost profiles on the EVM.

Architecture: Solidity for Algebra, Groth16 for Pairings

Stages 1-7 run natively in Solidity. The core operations — `addmod`, `mulmod`, and `keccak256` — are EVM primitives costing 8 and ~36 gas respectively. A sumcheck round costs roughly 90 gas. With ~300 rounds across all 7 stages, the sumcheck verification itself costs under 30K gas.

Stage 8 wraps the Dory opening verification in a Groth16 proof compiled via [gnark](#). The pairing check runs inside the SNARK circuit, and the Solidity contract verifies the Groth16 proof using the EVM’s `ecPairing` precompile (~316K gas).

The two halves connect through the Fiat-Shamir transcript: stages 1-7 in Solidity build the Keccak transcript state, then derive the Dory challenges that bind to the Groth16 proof’s public inputs. A forged proof would need to break either the sumcheck verification or the Groth16 soundness.

	Solidity (native EVM)		Groth16 (gnark circuit)
	+-----+		+-----+
Proof ----->	Stages 1-7		Stage 8
(calldata)	- Sumcheck verify	Keccak	- Dory evaluation
	- R1CS constraints	transcript	- Pairing check
	- Memory checking	----->	- MiMC public input
	- Lookup arguments	challenges	
	- Claim reductions		Verified via

```

+-----+ | ecPairing precompile |
+-----+

```

The Pipeline

Rust: Proof Export

The Jolt prover generates a standard proof, then an export layer (`onchain_export.rs`) serializes everything the Solidity verifier needs: preamble data, polynomial commitments, compressed univariate polynomials for each sumcheck round, intermediate evaluation claims, and the Dory witness.

The Dory witness feeds into a Go program that assigns a gnark circuit and generates a Groth16 proof. The circuit is parameterized by round count (7 rounds for small traces, 9 for production).

Solidity: Verification Contracts

The verifier is split across two contracts to fit within Ethereum's 24,576-byte contract size limit (EIP-170):

Contract	Size	Role
<code>JoltVerifierPhase1</code>	19,204 bytes	Preamble + commitments + stages 1-4
<code>JoltVerifier</code>	17,353 bytes	Stages 5-7 + stage 8 (Dory/Groth16)

Phase 1 returns the transcript state (`bytes32 state + uint32 nRounds`) to Phase 2, which resumes verification from the same Fiat-Shamir position.

Supporting libraries: - `JoltTranscript.sol` — Keccak Fiat-Shamir transcript (port of `jolt-core/src/transcripts/keccak.rs`) - `SumcheckVerifier.sol` — `evalFromHint` and round verification (assembly-optimized) - `BatchedSumcheckVerifier.sol` — Multi-instance batching with front-loaded power-of-2 scaling - `EqPolynomial.sol` — Multilinear extension evaluation - `UniSkipVerifier.sol` — Univariate skip optimization for stages 1-2 - `R1CSEvaluator.sol` — Lagrange basis R1CS constraint evaluation - `StageVerification.sol` — Per-stage verification logic (input claims, output claims, value aliasing) - `DoryOnChainVerifier.sol` — Groth16 wrapper with MiMC challenge hashing

gnark: Groth16 Dory Circuit

The Dory polynomial opening involves inner-product pairings that are expensive even with EVM precompiles. We compile a gnark circuit (`dory_verifier_9.go` for production) that performs the full Dory verification internally, exposing only a MiMC hash of the public inputs. The Solidity contract derives Dory challenges from the Keccak transcript, hashes them with MiMC, and checks the Groth16 proof against that hash.

Calldata Packing

A production SHA3 proof initially encoded to 133KB of calldata — over the default 128KB transaction size limit. ABI encoding of nested dynamic arrays (`bytes[]`, `uint256[][]`) adds 64 bytes of offset/length overhead per element.

Two packing optimizations brought it under budget:

1. **Commitment blob**: 42 polynomial commitments (each 384 bytes) packed into a single `bytes` field instead of `bytes[]`. Saves ~2.7KB by eliminating per-element ABI pointers.
2. **Dory message blob**: 113 transcript messages packed into `bytes` + packed `uint16` length index instead of `bytes[]`. Saves ~6.9KB.

Result: **133KB to 124KB** (9.6KB saved), comfortably under the 128KB limit.

On-Chain Results

Deployed and tested on Berachain Bepolia (chain ID 80069):

Proof	Program	Trace	Dory Rounds	Gas	Calldata
muldiv	Integer multiply-divide	1,024 steps	7	4,806,421	110KB
sha3	SHA3-256 hash	8,192 steps	9	5,332,785	124KB

Soundness verified: a corrupted proof (single flipped byte in a commitment) is rejected at **stage1: output claim mismatch** — the Fiat-Shamir transcript diverges immediately, and the sumcheck output claim check fails.

Gas Breakdown (approximate)

Component	Gas
Stages 1-7 sumcheck verification	~50K
Transcript operations (~300 keccak256)	~15K
Stage-specific claim logic	~40K
Groth16 ecPairing (stage 8)	~316K
Memory allocation and ABI decoding	~200K
Calldata cost (124KB x 16 gas/byte nonzero)	~2M
DELEGATECALL to Phase1	~2.7M

Calldata dominates. Further compression (e.g., EIP-4844 blob transactions) could reduce costs significantly.

Test Coverage

170 Foundry tests across 4 test contracts:

- **Algebraic tests:** Each stage verified independently with Rust-exported intermediate values
- **Transcript differential tests:** Solidity Keccak transcript state compared against Rust at every stage boundary, for both muldiv and sha3 proofs
- **Full E2E tests:** Complete proof verification through all 8 stages
- **Negative tests:** 40+ mutation tests — corrupted commitments, sumcheck coefficients, input claims, evaluation values, and stage proofs all correctly rejected
- **Edge case tests:** Zero-length inputs, boundary values, gas measurement

What's Next

- **InputHash alignment:** The Groth16 stage currently uses a mock verifier because the MiMC hash computed in Solidity doesn't yet match the gnark circuit's expected public input. Once aligned, the full pipeline runs with real Groth16 verification.
- **EIP-4844 blob transactions:** Moving proof data to blobs would cut calldata costs by ~10x.
- **Recursive composition:** Wrapping stages 1-7 in a SNARK for constant-size proofs.
- **Multi-program support:** The verifier is program-agnostic — any Jolt guest program's proof can be verified with the same contracts.

Deployed Contracts (Bepolia)

Contract	Address
MockGroth16Verifier	0xa4Df...926A
DoryOnChainVerifier	0x7A2D...926A
JoltVerifierPhase1	0x2E9A...e6D4
JoltVerifier	0x6Ae4...aa9

Sample Proofs Verified On-Chain

Proof	Program	Trace	Gas	Transaction
muldiv	Integer multiply-divide	1,024 steps	4,806,421	0x035e59...
sha3	SHA3-256 hash	8,192 steps	5,332,785	0x8fccbe...

Forged Proof Rejection

To demonstrate soundness, we took a valid SHA3 proof and flipped a single byte in the first polynomial commitment — the minimal possible corruption. We then submitted it to the deployed verifier.

The corrupted commitment poisons the Fiat-Shamir transcript from the very first hash operation. Every subsequent challenge the verifier derives is different from what the honest prover used, so the sumcheck output claim no longer matches. The verifier catches this at the earliest possible checkpoint:

```
Error: execution reverted: stage1: output claim mismatch
```

The Bepolia RPC node rejected the transaction during gas estimation itself — the forged proof cannot even enter the mempool as a pending transaction. This is the correct behavior: any modification to any proof element (commitments, sumcheck polynomials, evaluation claims, Groth16 proof) will cause a Fiat-Shamir divergence that the verifier detects and rejects.

The forged proof submission script is included at [foundry-test/script/SubmitFakeProof.s.sol](#).

Built with Jolt, Foundry, gnark, and a lot of field arithmetic.